

Bayesian modeling and inference with Stan

Modern Modeling Methods Conference

Fordham University, June 22, 2026

Mitzi Morris, Stan developer (mitzimorris on GitHub)

Bob Carpenter, Stan developer (bob-carpenter on GitHub)



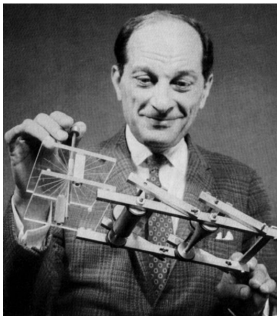
course materials:



Who is Stan?

- Stan is named in honor of **Stanislaw Ulam** (1909–1984)
- Co-inventor of the **Monte Carlo method**
- *Ulam holding the Fermiac, Enrico Fermi's physical Monte Carlo simulator for random neutron diffusion*

image from G. C. Geisler (2000) Los Alamos report LA-UR-2532



What is Stan?

- Stan is an **imperative probabilistic programming language**
 - cf., BUGS: declarative; Church, JAX: functional; PyMC/NumPyro: mix
- Stan **program**: defines a probability model
 - declares data and (constrained) parameter variables
 - defines log posterior (or penalized likelihood)
 - defines posterior predictive quantities (e.g., predictions, event probabilities)
- Stan **inference**: fits model to data & makes predictions
 - MCMC for full Bayesian inference
 - VB for approximate Bayesian inference
 - MLE for penalized maximum likelihood estimation

Warmup

Repeated trials

- Suppose we **repeatedly generate a random outcome** from among two or more potential outcomes
- Suppose the outcome **chances are the same each time**
 - i.e., outcomes are independent and identically distributed (i.i.d.)
- For example, **spin a fair spinner** (without cheating), such as one from *Family Cricket*.
- *This slide was generated in a cricket-playing country; please mentally substitute your favorite binary trial.*



Repeated binary trials

- Suppose the outcome is binary and assigned to 0 or 1; e.g.,
 - 20% chance of outcome 1: *ball in play*
 - 80% chance of outcome 0: *ball not in play*
 - *Substitute your favorite binary trial here, e.g., getting a test question right, reacting to a stimulus within a given time, etc.*
- How will **proportion of successes differ** given 10, 100, 1000, or 10000 trials?
 - Estimate ability as successes out of trials.
 - This is asking about the **standard error of the estimator**, which is the standard deviation of the estimate over varying data.

Simulated i.i.d. binary trials

- **10 trials** (10% to 50% success rate)
2 3 5 2 4 1 2 2 1 1
- **100 trials** (16% to 26% success rate)
26 18 23 17 21 16 21 15 21 26
- **1000 trials** (18% to 22% success rate)
181 212 175 213 216 179 223 198 188 194
- **10,000 trials** (19.3% to 20.3% success rate)
2029 1955 1981 1980 2001 2014 1931 1982 1989 2020

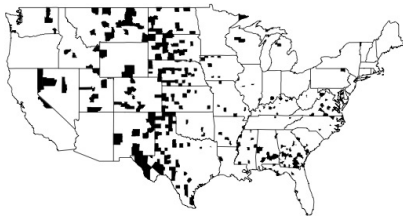
Stan code for simulating binary trials

```
transformed data {  
  array[4] int N = {10, 100, 1000, 10000};  
  real<lower=0, upper=1> theta = 0.46;  
}  
generated quantities {  
  array[4] int n = binomial_rng(N, theta);  
  vector[4] theta_hat = to_vector(n) ./ to_vector(N);  
}
```


Pop Quiz! Cancer Clusters

- Why do the lowest (left) and highest (right) cancer counties look so similar?

Lowest kidney cancer death rates



Highest kidney cancer death rates

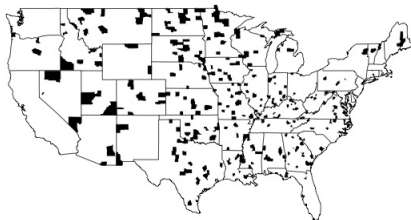


Image from Gelman et al., *Bayesian Data Analysis, 3rd Edition* (2013)

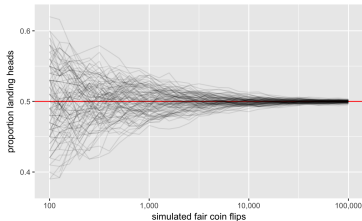
Pop Quiz Answer

- **Hint:** mix earlier simulations of repeated i.i.d. trials with 20% success and sort:

1/10	1/10	1/10	15/100	16/100
17/100	175/1000	179/1000	18/100	181/1000
188/1000	194/1000	198/1000	2/10	2/10
2/10	2/10	21/100	21/100	21/100
212/1000	213/1000	216/1000	223/1000	23/100
26/100	26/100	3/10	4/10	5/10

- More variation in observed rates with smaller sample sizes
- **Answer:** High cancer and low cancer counties are small populations

The central limit theorem



- **proportion heads** for **100 sequences** (light grey lines) of 100,000 flips
- **converges to expected value** 0.5 (red), with **standard deviation** $0.5/\sqrt{\# \text{ draws}}$
 - single flip standard deviation is 0.5
 - standard deviation is **standard error of maximum likelihood estimate**, $\hat{\theta} = \# \text{ heads} / \# \text{ flips}$

The central limit theorem (cont.)

- **The** theorem of statistics
 - Cardano (1501–1576) conjectured convergence; (Jacob) Bernoulli (1713) proved convergence for binomials (law of large numbers); de Moivre (1733) conjectured the CLT; Laplace (1812) proved i.i.d. version; Lyapunov (1901) removed i.i.d. constraint
- Sample **mean** of N independent variables with finite expectation/variance
 - **converges** to their expected mean as $N \rightarrow \infty$
 - relevant for **sampling-based Bayesian inference** as provided by Stan
 - **rate** of convergence is $\mathcal{O}\left(\frac{1}{\sqrt{N}}\right)$ with factor for standard deviation
 - i.e., each **decimal place of accuracy** requires **100× more draws**

What is Probability?

Probabilities quantify epistemic uncertainty

The curve described by a simple molecule of air or vapor is regulated in a manner just as certain as the planetary orbits; the **only difference** between them is that which comes from **our ignorance**.

Probability is relative, in part to this ignorance, in part **to our knowledge**.

—**Pierre-Simon Laplace, 1814**, *A Philosophical Essay on Probability*.

Every event is **in itself certain**, not probable; if we knew all, we should either know positively that it will happen, or positively that it will not.

We must remember that **the probability of an event is not a quality of the event itself**, but a mere name for the degree of ground which we, or some one else, have for expecting it.

... its probability to us means the **degree of expectation of its occurrence**, which we are warranted in entertaining by our present evidence.

—**John Stuart Mill, 1882**, *A System of Logic*. 8th Ed.

Random variables (a la Gelman and Hill)

- Imagine an urn filled with balls, each **recording the numerical value of every random variable**.
 - A random variable is a mapping from balls to values.
 - One ball represents the **real world**, but we don't know which.
- Or imagine **seeding a pseudorandom number generator**.
 - Given a **fixed seed**, the **sequence of pseudorandom numbers is deterministic**.
 - A pseudorandom variable is a mapping from seeds to values, e.g., `rnorm()` in R.

Examples of random variables

- the sum of the three best among a roll of four six-sided dice
- the time before the next traffic accident on a given highway
- the prevalence of a disease in a population
- the ability of a student to answer exam questions
- **Frequentist statistics** only uses random variables for **observable quantities** in a sequence, with probability defined as long-term sequence probability.
- **Bayesian statistics** uses random variables for **unobservable latent values, too.**

Probability is conditional on knowledge

- What is probability a man is **taller than 6'**?
 - What if I tell you he's Dutch?
 - What if I tell you he's a professional athlete?
 - What if I tell you he's a jockey? or a basketball player?
 - What if I tell you his mother is taller than 6'?
- What is the probability of **rain this afternoon** in Edinburgh?
 - What if I tell you it's April?
 - What if I tell you it has been raining all morning?

Events

- An **event** is a something that can occur or not occur.
- Events the main **bearers of probability** in statistics.
- An event is defined by **a property of random variables** (i.e., a subset of the balls in the urn), e.g.,
 - the prevalence of disease is greater than 2%, i.e., $\lambda > 0.02$
 - education scheme one is better than scheme two, i.e., $\theta_1 > \theta_2$
- The **probability of an event** is the probability that the random variables take on values that satisfy the property defining the event,
 - for the event $\lambda > 0.02$, the value $\Pr[\lambda > 0.02]$ is the probability that the random variable λ takes on a value greater than 0.02 (i.e., the probability that a random ball chosen from the urn has $\lambda > 0.02$).

Laplace's problem

aka, the first Bayesian posterior

Birth rate by sex

- **Laplace's** data on live births in Paris from 1745–1770:

<i>sex</i>	<i>live births</i>
female	241 945
male	251 527

- **Laplace's first question** (Estimation)
What is the birth rate of boys vs. girls?
- **Laplace's second question** (Event Probability)
Is a boy more likely to be born than a girl?
- Bayes (1763) set up the “Bayesian” model
- Laplace (1781, 1786) solved for the posterior

Bernoulli distribution

- The **Bernoulli distribution** has a binary outcome $y \in \{0, 1\}$, with 1 conventionally denoting “success.”
- The probability y is 1, written $\Pr[y = 1]$, is defined to be $\theta \in (0, 1)$, so

$$\text{Bernoulli}(y \mid \theta) = \begin{cases} \theta & \text{if } y = 1, \\ 1 - \theta & \text{if } y = 0, \text{ and} \\ 0 & \text{otherwise.} \end{cases}$$

- For example, if $y \in \{0, 1\}$ is a fair coin flip, we write

$$y \sim \text{Bernoulli}(0.5)$$

to indicate that the probability mass function of y is $\text{Bernoulli}(y \mid 0.5)$, which follows from $\Pr[y = 1] = 0.5$.

Binomial distribution

- In $\text{Binomial}(y \mid N, \theta)$, y is the **number of successes** in N **independent trials**, where each trial has a $\theta \in (0,1)$ **chance of success**.
- If $y_1, \dots, y_N \sim \text{Bernoulli}(\theta)$, then $(y_1 + \dots + y_N) \sim \text{Binomial}(N, \theta)$.
- The **analytic form** is

$$\text{Binomial}(y \mid N, \theta) = \binom{N}{y} \cdot \theta^y \cdot (1 - \theta)^{N-y},$$

where **binomial coefficient** $\binom{N}{y}$ is the **number of sequences** of 0s and 1s of length N with exactly y 1s.

Beta distribution

- **Beta distributions** are used to **model probabilities** or proportions.
- For parameters $\alpha, \beta > 0$ and a **continuous outcome** $\theta \in (0, 1)$,

$$\text{Beta}(\theta \mid \alpha, \beta) = \frac{1}{B(\alpha, \beta)} \cdot \theta^{\alpha-1} \cdot (1 - \theta)^{\beta-1}$$

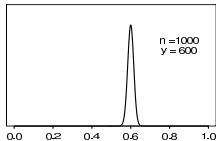
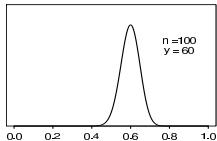
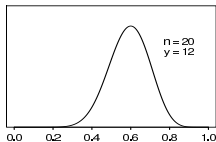
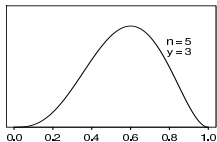
- Euler's **Beta function** $B(\alpha, \beta)$ is required to **normalize**.
- **Uniform is a beta**: $\text{Uniform}(\theta \mid (0, 1)) = \text{Beta}(\theta \mid 1, 1) =$
- **Conjugacy** (Laplace): With a beta prior and binomial data generating distribution, the posterior is a beta distribution.

Bayes's binomial model

- **Data**
 - y : total **number of male live births** (251,527)
 - N : total **number of live births** (493,472)
- **Parameter**: $\theta \in (0,1)$ for the **proportion of male** live births
- **Prior**: $\theta \sim \text{Beta}(1,1)$
- **Data generating distribution**: $y \sim \text{Binomial}(N, \theta)$
- **Joint density**: $p(y, \theta \mid N) = \text{Binomial}(y \mid N, \theta) \cdot \text{Beta}(\theta \mid 1, 1)$
- **Posterior density**: $p(\theta \mid y) = \text{Beta}(\theta \mid y + 1, N - y + 1)$

Example posteriors

- Unnormalized posterior density assuming uniform prior and y successes out of n trials (all with mean 0.6).



Estimation

- **Posterior** is $\text{Beta}(\theta \mid 1 + 241\,945, 1 + 251\,527)$
- Posterior **mean**:

$$\frac{1 + 251\,527}{1 + 241\,945 + 1 + 251\,527} \approx 0.509708 \underbrace{\quad}_{73}$$

- With uniform prior, **maximum likelihood estimate** is the **posterior mode**,

$$\frac{251\,527}{241\,945 + 251\,527} \approx 0.509708 \underbrace{\quad}_{82}.$$

- As number of observations approaches ∞ , posterior mean approaches MLE.

Event probability

- What is probability that a male live birth is more probable?

$$\begin{aligned}\Pr[\theta > 0.5] &= \int_{\Theta} I[\theta > 0.5] p(\theta \mid y, N) d\theta \\ &\approx 1 - 10^{-42} \\ &= 0.9999999999 \ 9999999999 \ 9999999999 \ 9999999999 \ 99.\end{aligned}$$

- $I[\phi] = 1$ if condition ϕ is true and 0 otherwise.
- Laplace solved Bayes's integral by
 - determining that the posterior was a beta distribution (conjugacy!)
 - and solving the normalization (gamma functions)

Mathematics vs. Simulation

- Luckily, we don't have to be as **good at math** as Laplace.
 - though math, probability, and computational fluency helps immensely with statistics!
- Nowadays, we **calculate integrals numerically** using tools like **Stan**.

If you wanted to do foundational research in statistics in the mid-twentieth century, you had to be bit of a mathematician, whether you wanted to or not. . . . if you want to do statistical research at the turn of the twenty-first century, you have to be a computer programmer.

—from Andrew's blog

Laplace's answer with Stan

```
transformed data {  
  int<lower=0> male = 251527;  
  int<lower=0> female = 241945;  
}  
parameters {  
  real<lower=0, upper=1> theta;  
}  
model {  
  male ~ binomial(male + female, theta);  
}  
generated quantities {  
  int<lower=0, upper=1> theta_gt_half = (theta > 0.5);  
}
```

And the answer is...

```
> fit <- stan("laplace.stan", iter=100000);  
> print(fit, probs=c(0.005, 0.995), digits=3)
```

	<i>mean</i>	<i>0.5%</i>	<i>99.5%</i>
<i>theta</i>	0.51	0.508	0.512
<i>theta_gt_half</i>	1.00	1.000	1.000

- Q1: θ is 99% certain to lie in $(0.508, 0.512)$.
 - i.e., $\Pr[\theta \in (0.508, 0.512) \mid \text{male, female}] = 0.99$.
- Q2: Laplace “morally certain” boys more prevalent.
 - i.e., $\Pr[\theta > 0.5] \approx 1$.

Bayesian workflow

Bayesian data analysis

- “By Bayesian data analysis, we mean practical methods for making inferences from data using probability models for quantities we observe and about which we wish to learn.”
- “The essential characteristic of Bayesian methods is their **explicit use of probability for quantifying uncertainty** in inferences based on statistical analysis.”

Bayesian workflow

- Set up **full probability model**
 - for **all** observable & unobservable quantities
 - consistent w. problem knowledge & data collection
- **Condition** on observed data (where Stan comes in)
 - to calculate posterior probability of unobserved quantities (e.g., parameters, predictions, missing data)
- **Evaluate** (where Stan sticks around)
 - model fit and implications of posterior
- **Repeat** as necessary

Ibid.

Priors, data distributions, and likelihoods

- We factor joint models into priors and data generating model (more later).
- The **prior** is what we know about latent parameters before we see the data.
- The **data generating model** generates observable data given parameters.
 - Considered as a function of the parameters for fixed data, the data generating model defines the **likelihood function**.
- Both priors and likelihoods are guided by our **knowledge**.
- Both prior and likelihood are **equally “subjective”**.
 - Conclusions are typically **way more sensitive to the likelihood**.
 - **“Don’t strain at the gnat of prior while swallowing the camel of the likelihood.”** (Andrew Gelmans’ blog)

Where do models come from?

- Sometimes model comes first, based on substantive considerations
 - toxicology, economics, ecology, physics, . . .
- Sometimes model chosen based on data collection
 - traditional statistics of surveys and experiments
- Other times the data comes first
 - observational studies, meta-analysis, . . .
- Usually its a mix

Model checking

- Do the inferences make sense?
 - are parameter values consistent with model's prior?
 - does simulating from parameter values produce reasonable fake data?
 - are marginal predictions consistent with the data?
- Do predictions and event probabilities for new data make sense?
- **Not:** Is the model true?
- **Not:** What is $\Pr[\text{model is true}]$?
- **Not:** Can we “reject” the model?

Model improvement

- Expanding the model
 - hierarchical and multilevel structure . . .
 - more flexible distributions (overdispersion, covariance)
 - more structure (geospatial, time series)
 - more modeling of measurement methods and errors
 - . . .
- Including more data
 - breadth (more predictors or kinds of observations)
 - depth (more observations)

Properties of Bayesian inference

- Explores full range of parameters consistent with prior info and data*
 - * if such agreement is possible
 - Stan automates this procedure with diagnostics
- Inferences can be plugged in directly for
 - parameter estimates minimizing expected error
 - predictions for future outcomes with uncertainty
 - event probability updates conditioned on data
 - risk assessment / decision analysis conditioned on uncertainty

All models are wrong, but some are useful

- This has to be one of the **highest impact section titles** ever!
- “Now it would be **very remarkable** if any system existing in the real world could be **exactly represented** by any simple model. However, cunningly chosen **parsimonious models** often do provide remarkably **useful approximations**.”
- **George Box** (1979)

Model (mis)specification

- A model is **well specified** if it matches the data-generating process of the data.
- All of our models will be **misspecified** to some extent
 - well specified: Newtonian physics (terrestrial)
 - mildly misspecified: Newtonian physics (solar scale)
 - wildly misspecified: Newtonian physics (near speed of light)
 - wildly misspecified: social science regression
- Models assumptions and predictions must be **tested**
 - ideally with **cross-validation** on quantities of interest

The folk theorem

“When you have computational problems, often there’s a problem with your model.”

— Andrew Gelman (2008)

- The usual culprits are
 - bugs in: samplers, data munging, model coding, etc.
 - model misspecification

http://andrewgelman.com/2008/05/13/the_folk_theore/

Model calibration

- Consider 100 days for which a meteorologist predicted a 70% chance of rain
 - about 70 of them should have had rain
 - not fewer, not more!
 - technically, expect $\text{Binomial}(100, 0.7)$ rainy day from a calibrated model
- Use posterior predictive checks to test calibration on
 - training data—can it fit?
 - held out data—can it predict?
 - cross-validation—approximates held out with training data
- Also applies to interval coverage of parameter values

Model sharpness

- Ideal forecasts are deterministic
 - predict 100% chance of rain or 0% chance of rain
 - always right
- A forecast of 90% chance of rain reduces uncertainty more than a 50% prediction
- A model is **sharp** if it has narrow posterior intervals
 - Prediction $\Pr[\alpha \in (1.2, 1.9)] = 0.9$
 - is sharper than $\Pr[\alpha \in (1, 2)] = 0.9$
- I.e., sharper models are more certain in its predictions
- Given calibration, we want our predictions to be **sharp**

Cross-validation

- Uses single data set to model held-out performance
- Assumes stationarity (as most models do)
- Partition data evenly into disjoint subsets (called **folds**)
 - 10 is a common choice
 - **leave-one-out** (LOO) uses a the number of training data points
- For each fold
 - estimate model on all data but that fold
 - test on that fold
- Usual comparison statistic is held out log likelihood

Bayesian Inference

Notation for Basic Quantities

- **Basic Quantities**

- y : observed data
- θ : parameters (and other unobserved quantities)
- x : constants, predictors for conditional (aka “discriminative”) models

- **Basic Predictive Quantities**

- $\tilde{y}$: unknown, potentially observable quantities
- $\tilde{x}$: constants, predictors for unknown quantities

Naming Conventions

- **Joint:** $p(y, \theta)$
- **Sampling / Likelihood:** $p(y|\theta)$
 - Sampling is function of y with θ fixed (prob function)
 - Likelihood is function of θ with y fixed (*not* prob function)
- **Prior:** $p(\theta)$
- **Posterior:** $p(\theta|y)$
- **Data Marginal (Evidence):** $p(y)$
- **Posterior Predictive:** $p(\tilde{y}|y)$

Bayes's Rule for Posterior

$$p(\theta|y) = \frac{p(y, \theta)}{p(y)} \quad \text{[def of conditional]}$$

$$= \frac{p(y|\theta) p(\theta)}{p(y)} \quad \text{[chain rule]}$$

$$= \frac{p(y|\theta) p(\theta)}{\int_{\Theta} p(y, \theta') d\theta'} \quad \text{[law of total prob]}$$

$$= \frac{p(y|\theta) p(\theta)}{\int_{\Theta} p(y|\theta') p(\theta') d\theta'} \quad \text{[chain rule]}$$

- **Inversion:** Final result **depends only** on **sampling distribution** (likelihood) $p(y|\theta)$ and **prior** $p(\theta)$

Bayes's rule up to proportion

- If data y is fixed, then

$$\begin{aligned} p(\theta|y) &= \frac{p(y|\theta) p(\theta)}{p(y)} \\ &\propto p(y|\theta) p(\theta) \\ &= p(y, \theta) \end{aligned}$$

- Posterior proportional to likelihood times prior.
- Equivalently, posterior proportional to joint.
- Avoids the nasty integral in the denominator for data marginal $p(y)$.

Posterior predictive distribution

- Predict new data $\tilde{y}$ based on observed data y
- Marginalize parameters θ out of posterior and likelihood

$$\begin{aligned} p(\tilde{y} \mid y) &= \mathbb{E}[p(\tilde{y}|\theta) \mid Y = y] \\ &= \int p(\tilde{y}|\theta) p(\theta|y) d\theta. \end{aligned}$$

- Weights predictions $p(\tilde{y}|\theta)$, by posterior $p(\theta|y)$
- Integral notation shorthand for sums and/or integrals

Posterior event probabilities

- Recall that an event A is a collection of outcomes
- So A may be defined by an indicator f on parameters

$$f(\theta) = \begin{cases} 1 & \text{if } \theta \in A \\ 0 & \text{if } \theta \notin A \end{cases}$$

- $f(\theta) = I(\theta_1 > \theta_2)$ for $\Pr[\theta_1 > \theta_2 | y]$,
 - $f(\theta) = I(\theta \in (0.50, 0.52))$ for $\Pr[\theta \in (0.50, 0.52) | y]$
- Defined by posterior expectation of indicator $f(\theta)$

$$\Pr[A | y] = \mathbb{E}[f(\theta) | y] = \int_{\Theta} f(\theta) p(\theta | y) d\theta.$$

Monte Carlo methods

for solving high-dimensional integrals

Monte Carlo methods

- Bayesian inference involves **high-dimensional integrals**.
- **Monte Carlo** methods **estimate** such integrals using probability.
- Specifically, suppose we want to calculate a **posterior expectation**,

$$\mathbb{E}[f(\theta) \mid y] = \int_{\Theta} f(\theta) \cdot p(\theta \mid y) \mathrm{d}y.$$

with respect to a distribution $\theta \sim p(\theta \mid y)$.

- Suppose we can **randomly sample** $\theta^{(m)} \sim p(\theta \mid y)$
- Then we can use a plug-in **Monte Carlo estimate**,

$$\mathbb{E}[f(\theta) \mid y] \approx \frac{1}{M} \sum_{m=1}^M f(\theta^{(m)}).$$

Stan samples from the posterior

- Stan's primary mode of operation is **sampling from a Bayesian posterior**
- Specifically, Stan will draw a **posterior sample** of size M ,

$$\theta^{(1)}, \dots, \theta^{(M)} \sim p(\theta \mid y).$$

- Stan uses **Markov chain Monte Carlo**
 - the sequence of draws is not independent
 - it usually has positive autocorrelation
 - in simple models, autocorrelation can be negative
- Specifically, the **no-U-turn sampler** (NUTS), a form of **Hamiltonian Monte Carlo**
 - NUTS/HMC has the best known scaling in dimension among generic samplers.

Parameter estimation

- Parameter estimates are posterior expectations (means)

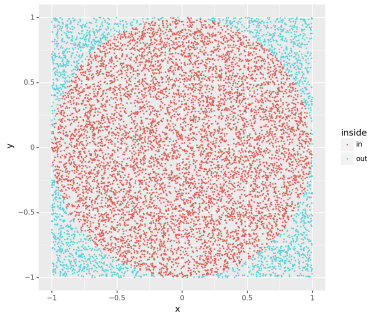
$$\hat{\theta} = \mathbb{E}[\theta \mid y] = \int \theta \cdot p(\theta \mid y) \mathrm{d}\theta \approx \frac{1}{M} \sum_{m=1}^M \theta^{(m)}$$

- Posterior mean **minimizes expected square error**.
- Posterior median **minimizes expected absolute error**.

Monte Carlo estimation of π

- Estimating $\pi = 3.14\dots$ via simulation is *the* textbook application of Monte Carlo methods.
- Generate points **uniformly** within the square.
- **Area of circle** is:
proportion within circle times square's area.
- **Circle's area** is π ; **square's area** is 4.
 - radius $r = 1$ yields area $\pi \cdot r^2 = \pi$
- **If** $X, Y \sim \text{Uniform}((-1, 1))$, **then**

$$4 \cdot \Pr[X^2 + Y^2 \leq 1] = \mathbb{E}\left[4 \cdot \mathbf{I}[X^2 + Y^2 \leq 1]\right] \\ = \pi.$$



Calculating π with Stan

```
generated quantities {  
  real<lower=-1, upper=1> x = uniform_rng(-1, 1);  
  real<lower=-1, upper=1> y = uniform_rng(-1, 1);  
  int<lower=0, upper=1> in_circle = hypot(x, y) <= 1;  
  int<lower=0, upper=4> pi = 4 * in_circle;  
}
```

- **Simulate** X and Y
- **Indicator** `in_circle` is 1 if (x, y) is in the circle, 0 otherwise.
 - Stan's hypoteneuse function: `hypot(a, b) = sqrt(a^2 + b^2)`
- **Multply by 4** to get `pi`, which over multiple draws has **expectation** π .

Fitting Stan model with CmdStanPy

```
m = csp.CmdStanModel(stan_file = 'pi.stan')
s = m.sample(iter_sampling=100_000)
s.summary() # trimmed by hand
```

	<i>Mean</i>	<i>MCSE</i>
<i>pi</i>	3.142900	0.002599

- Our **estimate** is $\hat{\pi} = 3.143$
- 0.0026 is our **standard error** (i.e., standard deviation of estimator error)
- Rough **95% confidence interval** is $\text{mean} \pm 2 \cdot \text{mcse} = (3.138, 3.148)$.
- True value $\pi = 3.141 \dots$ within that interval.

Accuracy of Markov chain Monte Carlo

- Monte Carlo Integration computes the exact posterior to within any ϵ
 - i.e., it is *not* an approximation of the posterior like variational Bayes or a Laplace approximation
- Monte Carlo draws are i.i.d. by definition, but MCMC draws are not
- Central limit theorem extends to MCMC: expected error decreases at rate of $\frac{1}{\sqrt{N}}$.
- Need $100\times$ larger sample for each digit of accuracy

Parameter (co)variance estimation

- Posterior **parameter variance** estimates for univariate θ are

$$\begin{aligned}\text{var}[\theta \mid y] &= \mathbb{E}\left[(\theta - \hat{\theta})^2\right] \\ &\approx \frac{1}{M} \sum_{m=1}^M \left(\theta^{(m)} - \hat{\theta}\right)^2\end{aligned}$$

- Posterior **parameter covariance** estimation for multivariate θ is similar,

$$\begin{aligned}\text{cov}[\theta \mid y] &= \mathbb{E}\left[(\theta - \hat{\theta}) \cdot (\theta - \hat{\theta})^\top\right] \\ &\approx \frac{1}{M} \sum_{m=1}^M \left(\theta^{(m)} - \hat{\theta}\right) \cdot \left(\theta^{(m)} - \hat{\theta}\right)^\top.\end{aligned}$$

Posterior event probabilities

- Posterior **event probabilities** are computed similarly

$$\begin{aligned}\Pr[\text{cond}(\theta) \mid y] &= \int \mathbb{I}[\theta \in A] \cdot p(y \mid \theta) \, d\theta \\ &\approx \frac{1}{M} \sum_{m=1}^M \mathbb{I}[\theta^{(m)} \in A].\end{aligned}$$

- The **indicator function** $\mathbb{I}[\theta \in A]$ evaluates to 1 if θ satisfies the event A , and evaluates to 0 otherwise.
 - i.e., *event probabilities are expectations of indicator functions.*

Posterior predictive densities

- Evaluating the **posterior predictive density** is another expectation,

$$\begin{aligned} p(\tilde{y} \mid y) &= \mathbb{E}[p(\tilde{y}) \mid y] \\ &\approx \frac{1}{M} \sum_{m=1}^M p(\tilde{y} \mid \theta^{(m)}) \end{aligned}$$

- Sampling** from the posterior predictive requires random number generation,

$$\tilde{y}^{(m)} \sim p(\tilde{y} \mid \theta^{(m)})$$

- Two forms of uncertainty** compound for posterior predictive,
 - **estimation uncertainty** for θ given data y represented by $\theta^{(m)} \sim p(\theta \mid y)$
 - **data generation uncertainty** due to sampling $\tilde{y}^{(m)} \sim p(\tilde{y} \mid \theta^{(m)})$

Posterior quantiles

- Posterior **quantiles** (e.g., **intervals** and **median**) just use the empirical quantiles of the draws,

$$\text{quantile}(\theta, \alpha) = \text{sort}(\theta^{(1:M)}) \left[\lceil \alpha \times M \rceil \right]$$

- As α approaches 0 or 1, the number of draws beyond α shrinks and **estimation uncertainty increases dramatically**.
 - e.g., a 2.5% quantile defining the ends of a central 95% interval has around 2.5 times as much uncertainty (coincidental that it's 2.5 and 2.5%) as a mean.

Bayesian comparisons

Bayesian “Fisher Exact Test”

- Suppose we observe the following **data on handedness vs. sex**.

	<i>sinister</i>	<i>dexter</i>	TOTAL
<i>male</i>	9 (y_1)	43	52 (N_1)
<i>female</i>	4 (y_2)	44	48 (N_2)

- Assume **data generating model** $\text{Binomial}(y_k \mid N_k, \theta_k)$, uniform priors
- Scientific question:** Are men more likely to be lefthanded?

$$\begin{aligned}\Pr[\theta_1 > \theta_2 \mid y, N] &= \int_{\Theta} \mathbb{I}[\theta_1 > \theta_2] p(\theta \mid y, N) d\theta \\ &\approx \frac{1}{M} \sum_{m=1}^M \mathbb{I}[\theta_1^{(m)} > \theta_2^{(m)}].\end{aligned}$$

Stan Binomial Comparison

```
data {  
  int y[2];  
  int N[2];  
}  
parameters {  
  vector<lower=0,upper=1> theta[2];  
}  
model {  
  y ~ binomial(N, theta);  
}  
generated quantities {  
  real boys_minus_girls = theta[1] - theta[2];  
  int boys_gt_girls = theta[1] > theta[2];  
}
```

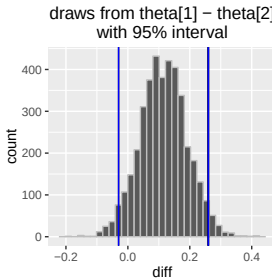
Binomial Comparison Results

	<i>mean</i>	<i>2.5%</i>	<i>97.5%</i>
<i>theta[1]</i>	0.22	0.12	0.35
<i>theta[2]</i>	0.11	0.04	0.21
<i>boys_minus_girls</i>	0.12	-0.03	0.26
<i>boys_gt_girls</i>	0.93	0.00	1.00

- $\Pr[\theta_1 > \theta_2 \mid y] \approx 0.93$
- $\Pr[(\theta_1 - \theta_2) \in (-0.03, 0.26) \mid y] = 95\%$

Visualizing Posterior Difference

- Plot of posterior difference, $p(\theta_1 - \theta_2 \mid y, N)$ (men - women)



- Vertical bars: central 95% posterior interval $(-0.03, 0.26)$

Generalized linear models

Linear regression

```
data {  
  int<lower=0> N;           // # observations  
  vector[N] x;             // covariates  
  vector[N] y;             // outcomes  
}  
parameters {  
  real alpha;              // intercept  
  real beta;               // slope  
  real<lower=0> sigma;      // error scale  
}  
model {  
  {alpha, beta} ~ normal(0, 5); // prior  
  sigma ~ lognormal(0, 1);      // prior  
  y ~ normal(alpha + beta * x, sigma); // likelihood  
}
```

Distribution statements

- Distribution statements are vectorized, so these two are the same:

```
y ~ normal(alpha + beta * x, sigma);
```

```
for (n in 1:N)  
  y[n] ~ normal(alpha + beta * x[n], sigma);
```

- These two target increment statements are also the same as above:

```
target += normal_lupdf(y | alpha + beta * x, sigma);
```

```
for (n in 1:N)  
  target += normal_lupdf(y[n] | alpha + beta * x[n], sigma);
```

Stan's constrained parameters

- Scale `sigma` declared `real<lower=0>` with prior `sigma ~ lognormal(0, 1)`.
 - Stan provides **uniform** distribution over parameters **satisfying constraints**.
 - **user-defined priors** multiply-in as **factors**.
 - $\sigma \in (0, \infty)$ is **log-transformed** to $\tau = \log \sigma \in (-\infty, \infty)$ for **sampling**.
 - adjust for **change-of-variables** from σ to τ

$$\begin{aligned} p(\tau) &= p(\exp(\tau)) \cdot \left| \frac{d}{d\tau} \exp(\tau) \right| \\ &= p(\sigma) \cdot \exp(\tau) \\ &= \text{lognormal}(\exp(\tau) \mid 0, 1) \cdot \exp(\tau). \end{aligned}$$

Posterior predictive inference

- Given “training” data (x, y) and fresh covariates x^{new} , **sample** from the **posterior predictive distribution**, $y^{\text{new}(m)} \sim p_{Y^{\text{new}}|x^{\text{new}}, x, y}$.
- Include new covariates as **additional data variables**.

```
data {  
  ...  
  int<lower=0> N_new;      // # predictions  
  vector[N_new] x_new;    // covariates for predictions  
}
```

- Use a random number generator in **generated quantities block**.

```
generated quantities {  
  vector[N_new] y_new = to_vector(normal_rng(alpha + beta * x_new, sigma));  
}
```

Poisson regression

- Swap out data declaration and likelihood for Poisson.

```
data {  
  int<lower=0> N;           // # obs  
  vector[N] x;             // covariates  
  array[N] int<lower=0> y;  // count outcomes  
}  
parameters {  
  real alpha;              // intercept  
  real beta;               // slope  
}  
model {  
  { alpha, beta } ~ normal(0, 5); // priors  
  y ~ poisson_log(alpha + beta * x); // likelihood  
  // note: poisson_log(mu) = poisson(exp(mu))  
}
```

Log odds

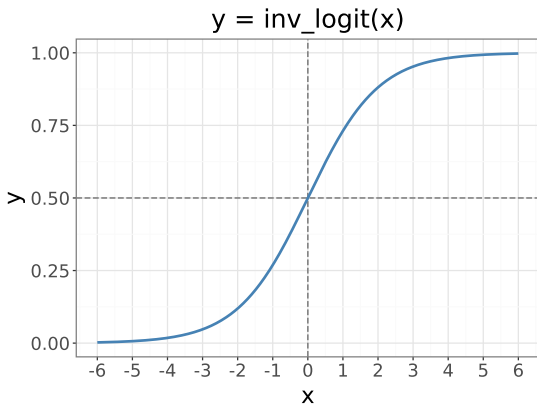
- **Logistic regression** is a linear regression on the log odds of a binary outcome.
- A probability $p \in (0, 1)$ has **odds** $p : (1 - p)$ and **odds ratio** $\frac{p}{1 - p}$.
- The **log odds**, conventionally denoted *logit*, and its inverse,

$$\text{logit}(p) = \log \frac{p}{1 - p}$$

$$\text{logit}^{-1}(v) = \frac{\exp(v)}{1 + \exp(v)}.$$

p	odds	odds ratio	log odds	note
1/4	1:3	$(1/4) / (3/4) = 1/3$	$-\log 3$	log odds negative if $p < 0.5$
1/2	1:1	$(1/2) / (1/2) = 1$	0	log odds 0 if $p = 1/2$
3/4	3:1	$(3/4) / (1/4) = 3$	$\log 3$	$\text{logit}(p) = -\text{logit}(1 - p)$ symmetric

Logistic sigmoid: $y = \text{logit}^{-1}(x) = \frac{\exp(x)}{1+\exp(x)}$



Logistic regression

- You guessed it—swap for logistic

```
data {  
  int<lower=0> N;           // # obs  
  vector[N] x;             // covariates  
  array[N] int<lower=0, upper=1> y; // binary outcomes  
}  
parameters {  
  real alpha;               // intercept  
  real beta;                // slope  
}  
model {  
  {alpha, beta} ~ normal(0, 5); // prior  
  y ~ bernoulli_logit(alpha + beta * x); // log odds: alpha + beta * x  
}
```

Multiple regression

- Data $N \times K$ matrix (w. column of intercepts); slopes K -vector.

```
data {  
  int<lower=0> N;  
  int<lower=1> K;           // # covariates per outcome  
  matrix[N, K] x;         // data matrix  
  array[N] int<lower=0> y;  
}  
parameters {  
  vector[K] beta;          // K slopes  
}  
model {  
  beta ~ normal(0, 5);      // vectorized prior, following likelihood  
  y ~ poisson_log(x * beta); // vec-mat multiply (x * beta) is N-vector  
}
```

Autoregressive models

- A **times series** is just a sequence of points $v_1, v_2, \dots, v_T$.
- An **AR(1)** model regresses on the previous point in time for $t > 1$,

$$v_t \sim \text{normal}(\alpha + \beta \cdot v_{t-1}, \sigma).$$

- If $\beta \in (-1, 1)$, then as $t \rightarrow \infty$, the **stationary** distribution is

$$v_t \sim \text{normal}\left(\frac{\alpha}{1 - \beta}, \frac{\sigma}{1 - \beta^2}\right).$$

- An **AR(k)** model is just a linear regression on the previous k points.

Autoregression in Stan

- We can have an autoregressive data distribution or prior; either way

```
vector[T] v;                                     // data or param

real alpha; real<lower=0, upper=1> beta;          // stationary param
v[1] ~ normal(alpha / (1 - beta),                 // marginal
              sigma / sqrt(1 - beta^2));
v[2:T] ~ normal(alpha + beta * v[1:(T - 1)], sigma); // AR(1)
```

- Last line is equivalent to

```
for (t in 2:T)
  v[t] ~ normal(alpha + beta * v[t-1], sigma);
```


Crowdsourcing

Bradley-Terry crowdsourcing model

- Bradley and Terry (early 1950s) collected data on **consumers comparing pairs of products** to estimate their quality.
- Products j are parameterized with a **real valued quality** $\alpha_j \in (-\infty, \infty)$.
- The n -th rating compares **products** i_n and j_n , with **rating** $y_n = 1$ if product i_n **preferred** to j_n
- Log odds of i_n being better than j_n is $\alpha_{i[n]} - \alpha_{j[n]}$,

$$y_n \sim \text{bernoulli_logit}(\alpha_{i[n]} - \alpha_{j[n]}).$$

- Elo (1960s) rediscovered a rescaled variant and proposed on online update
 - update is via stochastic gradient descent (Robbins and Monro, early 1950s).

Bradley-Terry code

```
data {  
  int<lower=0> N;           // # matchups  
  int<lower=0> J;           // # items  
  array[N] int<lower=1, upper=J> ii; // first item  
  array[N] int<lower=1, upper=J> jj; // second item  
  array[N] int<lower=0, upper=1> y;  // outcomes  
}  
parameters {  
  vector[J] alpha;          // abilities (log odds scale)  
}  
model {  
  alpha ~ normal(0, 5);      // prior  
  y ~ bernoulli_logit(alpha[ii] - alpha[jj]); // vectorized likelihood  
  
  // equiv for (n in 1:N) y[n] ~ bernoulli_logit(alpha[ii[n]] - alpha[jj[n]]);  
}
```

Item-response theory (1PL)

- Used for **educational test design**.
- Response y_n is 1 if student j_n **correctly answered** test question i_n .
- Like Bradley-Terry, w. student **abilities** α_j and question **difficulties** β_i ,

$$y_n \sim \text{bernoulli}(\text{logit}^{-1}(\alpha_{j[n]} - \beta_{i[n]})).$$

- In Stan,

```
vector[J] alpha; vector[I] beta;  
...  
y ~ bernoulli_logit(alpha[jj] - beta[ii]);
```

- The **one-parameter logistic** (1PL) involves difficulty parameter with ability being a “nuisance” required to match data.

Item-response theory (2PL)

- Want to know how **discriminative** test items are
 - an item is discriminative if abilities greater than difficulty have high chance of getting it correct and abilities lower than difficulty have low chance
- Add discrimination $\delta_i > 0$ for items, and use

$$y_n \sim \text{bernoulli}(\text{logit}^{-1}(\delta_{i[n]} \cdot ((\alpha_{j[n]} - \beta_{i[n]}))).$$

- As $\delta_i \rightarrow \infty$, if $\beta_j < \alpha_i$, then log odds go to $-\infty$ and if $\beta_j > \alpha_i$, log odds go to ∞ .

```
vector[I] delta;
```

```
...
```

```
y ~ bernoulli_logit(delta .* (alpha[jj] - beta[ii])); // .* elementwise
```

Hierarchical Models

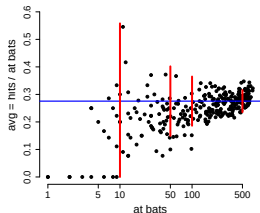
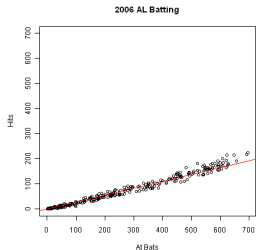
for repeated binary trials

Baseball batting ability

- Batters have a number of “at-bats” in a season, out of which they get a number of “hits” (hits are a good thing)
 - details not relevant
 - **same approach applies to clinical trials** (BDA), pediatric surgery complications (BUGS), reviews on Yelps, tests in multiple classrooms, ...
- No player with higher than 40% success rate since 1950s
 - and then only *slightly higher* than 40% with *low at-bats*
- No player (excluding “bowlers”) has success rate much less than 20%.

Baseball data

- (top) **Hits vs. at-bats** for 2006 American League (excluding pitchers)
- (bottom) **Average vs. at-bats**
 - blue line is pooled average (0.271)
 - red is ± 2 binomial std devs
- **Not much variation**; it's related to trials
- **Success rate increases** with number of trials
 - at-bats should be a covariate!



Pooling data (aka sharing strength)

- **How do we estimate** with a **small number of trials**? e.g., 6 hits in 10 at bats, or 0 in 5?
 - maximum likelihood estimates of 0.6 or 0.0 are **absurd**.
- Same logic applies to players with 40 hits in 131 at bats or 152 hits in 537.
- **No pooling**: estimate each player separately (assume no similarities)
- **Complete pooling**: estimate all players together (assume no differences)
- **Partial pooling**: somewhere in the middle (let data decide)
 - use information about other players (i.e., the population) as warranted by the variation in the data

Complete pooling model

- Assume players all have **same ability** ϕ .
 - implicit uniform prior

```
data {  
  int<lower=0> N;           // items  
  array[N] int<lower=0> K;  // trials  
  array[N] int<lower=0> y;  // successes  
}  
parameters {  
  real<lower=0, upper=1> phi; // chance of success  
}  
model {  
  y ~ binomial(K, phi);      // likelihood  
}
```

No pooling model code

- Assume each player has independent ability $\phi[n]$.
 - implicit uniform prior

```
data {  
  int<lower=0> N;  
  array[N] int<lower=0> K;  
  array[N] int<lower=0> y;  
}  
parameters {  
  vector<lower=0, upper=1>[N] phi;  
}  
model {  
  y ~ binomial(K, phi);  
}
```

Hierarchical Models

- Hierarchical models principled way to **determine how much pooling**.
 - Alternative is to stipulate all or nothing in likelihood (cf. “subjective”).
- **Pull estimates** toward the **population mean** based on amount of **variation** in population
 - **low variance** population: more pooling
 - **high variance** population: less pooling
- **In limit**
 - as variance goes to 0, get complete pooling
 - as variance goes to ∞ , get no pooling

Hierarchical Batting Ability

- Instead of fixed priors, **estimate priors** along with other parameters
- Still only **uses data once** for a single model fit (i.e., not empirical Bayes)
- **Data**: y_n, K_n : hits, at-bats for player n
- **Parameters**: ϕ_n : ability for player n
- **Hyperparameters**: α, β : population mean and variance
- **Hyperpriors**: fixed (constant) priors on α and β

Hierarchical Batting Model (cont.)

$$\theta \sim \text{Uniform}(0,1)$$

$$\kappa \sim \text{Pareto}(1.5)$$

$$\phi_n \sim \text{Beta}(\kappa \theta, \kappa (1 - \theta))$$

$$y_n \sim \text{Binomial}(K_n, \phi_n)$$

- θ is prior mean; κ is prior count (plus 2).
- **Pareto** provides **power law** distribution on prior count:

$$\text{Pareto}(u \mid \alpha) \propto \frac{\alpha}{u^{\alpha+1}}$$

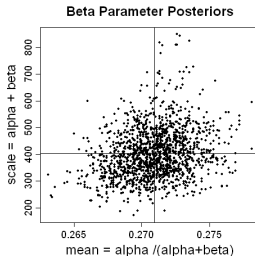
- Could (*should!*) use more informative prior on hyperparameter θ .

Partial Pooling Model in Stan

```
data {  
  int<lower=0> N;  
  array[N] int<lower=0> K;  
  array[N] int<lower=0> y;  
}  
parameters {  
  real<lower=0, upper=1> theta;  
  real<lower=1> kappa;  
  vector<lower=0, upper=1>[N] phi;  
}  
model {  
  kappa ~ pareto(1, 1.5);           // hyperprior  
  theta ~ beta(kappa * theta,      // prior  
               kappa * (1 - theta));  
  y ~ binomial(K, theta);           // likelihood  
}
```

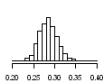
Posterior for Hyperpriors

- **Scatterplot of draws**
- Crosshairs at **mean**
- $\kappa = \alpha + \beta$ and $\theta = \frac{\alpha}{\alpha + \beta}$
- **Prior mean** estimate: $\hat{\theta} = 0.271$
- **Prior count** estimate: $\hat{\kappa} = 400$
- Together yield **prior standard deviation** of only 0.022!!!

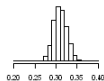


Posterior Ability (High Avg Players)

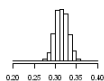
22 / 60



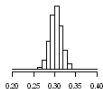
165 / 482



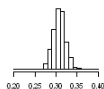
181 / 521



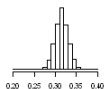
224 / 695



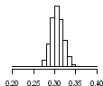
214 / 648



214 / 623



200 / 607



Who's the Best?

- **Posterior probability** that player n has **highest ability** is

$$\Pr[\phi_n \geq \max(\phi) \mid y]$$

- Code event with **indicator variable** in Stan

```
generated quantities {  
  int<lower=0, upper=1> is_best[N];  
  real max_phi = max(phi); // compute this just once  
  for (n in 1:N)  
    is_best[n] = phi[n] >= max_phi;  
}
```

Multiple Comparisons

- Hierarchical model **adjusts for multiple comparisons** by pulling all estimates toward population mean
- cf. Gelman, A., Hill, J. and Yajima, M., 2012. Why we (usually) don't have to worry about multiple comparisons. *Journal of Research on Educational Effectiveness*, 5(2):189–211.

Results for 2006 AL Season

<i>Player</i>	<i>Average</i>	<i>At-Bats</i>	<i>Pr[best]</i>
Mauer	.347	521	0.12
Jeter	.343	623	0.11
	.342	482	0.08
	.330	648	0.04
	.330	607	0.04
	.367	60	0.02
	.322	695	0.02

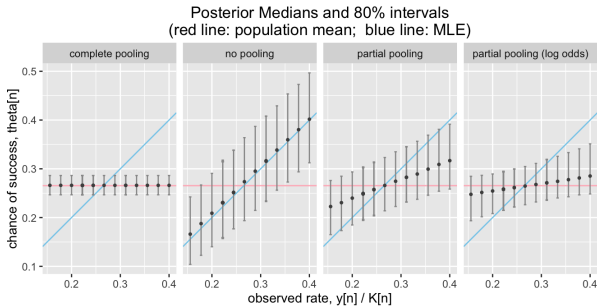
- Posterior probabilities reflect uncertainty in data
- In the **last game of the season** (out of 162), **Joe Mauer** (Minnesota Twins) edged out **Derek Jeter** (NY Yankees).
- It's **a lot of luck**; sorry for **ruining your enjoyment** of sports statistics.

Efron & Morris (1975) Data

- From their classic analysis for shrinkage/empirical Bayes
- Picked batters with 45 at bats on a given day (artificial!)

	FirstName	LastName	Hits	At.Bats	Rest.At.Bats	Rest.Hits
1	Roberto	Clemente	18	45	367	127
2	Frank	Robinson	17	45	426	127
3	Frank	Howard	16	45	521	144
4	Jay	Johnstone	15	45	275	61
5	Ken	Berry	14	45	418	114
6	Jim	Spencer	14	45	466	126
7	Don	Kessinger	13	45	586	155
8	Luis	Alvarado	12	45	138	29
9	Ron	Santo	11	45	510	137
10	Ron	Swaboda	11	45	200	46
11	Rico	Petrocelli	10	45	538	142

Pooling vs. No-Pooling Estimates

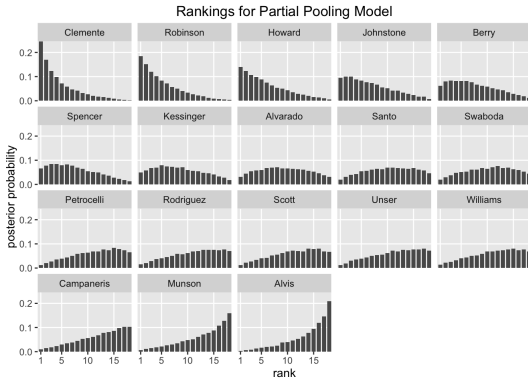


- complete pooling, no pooling, partial pooling, (log odds)

Ranking

```
generated quantities {  
  int<lower=1, upper=N> rnk[N];      // rank of player n  
  {  
    int dsc[N];  
    dsc = sort_indices_desc(theta);  
    for (n in 1:N)  
      rnk[dsc[n]] = n;  
  }  
}
```

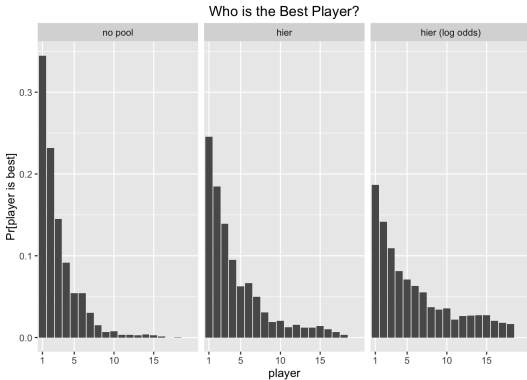
Posterior Ranks



Who is Best? better Stan code

```
generated quantities {  
  ...  
  int<lower=0, upper=1> is_best[N];  
  ...  
  for (n in 1:N)  
    is_best[n] = (rnk[n] == 1); // more efficient  
  ...  
}
```

Who is Best? Posterior



Posterior Predictive Inference

- How do we predict new outcomes (e.g., rest of season)?

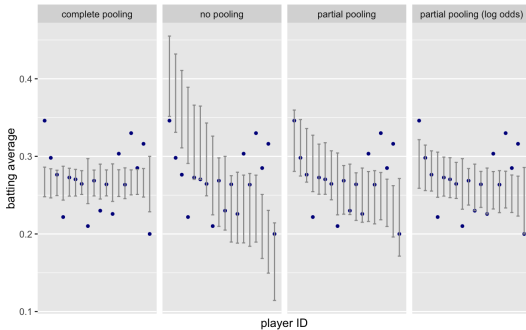
```
data {  
  int<lower=0> K_new[N];      // new trials  
  int<lower=0> y_new[N];      // new outcomes  
  ...  
generated quantities {  
  int<lower=0> z[N]; // posterior prediction  
  for (n in 1:N)  
    z[n] = binomial_rng(K_new[n], theta[n]);  
}
```

- Full Bayes accounts for two sources of uncertainty
 - estimation uncertainty (built into posterior)
 - sampling uncertainty (explicit RNG function)

Posterior Predictions

Posterior Predictions for Batting Average in Remainder of Season

50% posterior predictive intervals (gray bars); observed (blue dots)

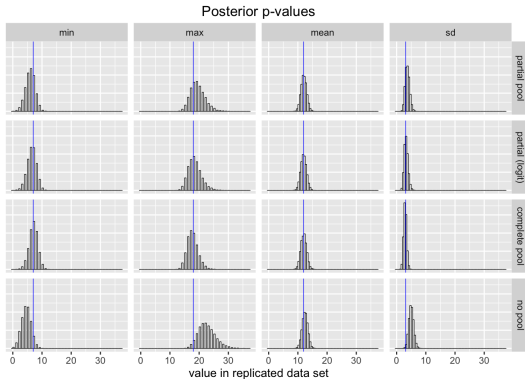


Posterior Predictive Check

- Replicate data from parameters

```
generated quantities {  
  ...  
  for (n in 1:N)  
    y_rep[n] = binomial_rng(K[n], theta[n]);  
  for (n in 1:N)  
    y_pop_rep[n] = binomial_rng(K[n],  
                                beta_rng(phi * kappa,  
                                          (1 - phi) * kappa));  
  
  min_y_rep = min(y_rep);  
  sd_y_rep = sd(to_vector(y_rep));  
  p_min = (min_y_rep >= min_y);  
  p_sd = (sd_y_rep >= sd_y);  
}
```

Posterior p -Values



Calibration and Sharpness

- **Calibration:** A model is calibrated if the 50% intervals contain roughly 50% of the true intervals
 - technically, we expect $\text{Binomial}(N, 0.5)$ of N parameters to fall in their 50% intervals
 - we can evaluate with held-out data using cross-validation
- **Sharpness:** One posterior is sharper than another if it concentrates more posterior mass around the true value
 - e.g., central posterior intervals of interest are narrower
 - see: Gneiting, Balabdaoui, and Raftery (2007) Probabilistic forecasts, calibration and sharpness. *JRSS B*.

More in the Case Study

- This talk roughly followed my Stan case study:
 - Hierarchical Partial Pooling for Repeated Binary Trials
- Available under case studies at
 - <http://mc-stan.org/documentation>.

The Stan Language

Stan is a programming language

- **Not** a graphical specification language like BUGS or JAGS
- Stan is a Turing-complete imperative programming language **for specifying differentiable log densities** and **posterior predictive inferences**.
 - reassignable local variables and scoping
 - full conditionals and loops
 - functions (including recursion)
- With automatic “black-box” inference on top (though even that is tunable)
- Programs computing same thing may have different efficiency

Basic program blocks

- **data** (once)
 - *content*: declare data types, sizes, and constraints
 - *execute*: read from data source, validate constraints
- **parameters** (every log prob eval)
 - *content*: declare parameter types, sizes, and constraints
 - *execute*: transform to constrained, Jacobian
- **model** (every log prob eval)
 - *content*: statements defining posterior density
 - *execute*: execute statements

Derived variable blocks

- **transformed data** (once after data)
 - *content*: declare and define transformed data variables
 - *execute*: execute definition statements, validate constraints
- **transformed parameters** (every log prob eval)
 - *content*: declare and define transformed parameter vars
 - *execute*: execute definition statements, validate constraints
- **generated quantities** (once per draw, double type)
 - *content*: declare and define generated quantity variables;
includes pseudo-random number generators
(for posterior predictions, event probabilities, decision making)
 - *execute*: execute definition statements, validate constraints

Reading and transforming data

- Only done once for optimization or sampling (per chain)
- Read data
 - read data variables from memory or file stream
 - validate data
- Generate transformed data
 - execute transformed data statements
 - validate variable constraints when done

Evaluating the log density

- *Given* parameter values on unconstrained scale
- Builds expression graph for log density (start at 0)
- Inverse transform parameters to constrained scale
 - constraints involve non-linear transforms
 - e.g., positive constrained x to unconstrained $y = \log x$
- account for curvature in change of variables
 - e.g., unconstrained y to positive $x = \log^{-1}(y) = \exp(y)$
 - e.g., add log Jacobian determinant, $\log \left| \frac{d}{dy} \exp(y) \right| = y$
- Execute model block statements to increment log density

Evaluating gradients

- Log density evaluation builds up expression graph
 - templated overloads of functions and operators
 - efficient arena-based memory management
- Compute gradient in backward pass on expression graph
 - propagate partial derivatives via chain rule
 - work backwards from final log density to parameters
 - dynamic programming for shared subexpressions
- Linear multiple of time to evaluate log density

Evaluating generated quantities

- **Given** parameter values
- Once per iteration (not once per leapfrog step)
- May involve (pseudo) random-number generation
 - Executed generated quantity statements
 - Validate values satisfy constraints
- Typically used for
 - Event probability estimation
 - Predictive posterior estimation
- Efficient because evaluated with double types (no autodiff)

Variable transforms

- Code HMC and optimization with $\mathbb{R}^n$ **support**
- Transform constrained parameters to unconstrained
 - lower (upper) bound: offset (negated) log transform
 - lower and upper bound: scaled, offset logit transform
 - simplex: centered, stick-breaking logit transform
 - ordered: free first element, log transform offsets
 - unit length: spherical coordinates
 - covariance matrix: Cholesky factor positive diagonal
 - correlation matrix: rows unit length via quadratic stick-breaking

Variable transforms (cont.)

- Inverse transform from unconstrained $\mathbb{R}^n$
- Evaluate log probability in model block on natural scale
- Optionally adjust log probability for change of variables
 - adjustment for MCMC and variational, not MLE
 - add log determinant of inverse transform Jacobian
 - automatically differentiable

Variable and expression types

Variables and expressions are **strongly, statically typed**.

- **Primitive:** int, real
- **Matrix:** matrix[M,N], vector[M], row_vector[N]
- **Bounded:** primitive or matrix, with
 <lower=L>, <upper=U>, <lower=L,upper=U>
- **Constrained Vectors:** simplex[K], ordered[N], pos_ordered[N], sum_to_zero[K]
 positive_ordered[N], unit_length[N]
- **Constrained Matrices:** cov_matrix[K], corr_matrix[K], cholesky_factor_cov[M,N],
 cholesky_factor_corr[K], row_stochastic_matrix[M, N], column_stochastic_matrix[M,
 N],
- **Arrays:** of any type (and dimensionality) array[N] T

Integers and reals

- Different types (conflated in BUGS, JAGS, and R)
- Distributions and assignments care
- Integers may be assigned to reals but not vice-versa
- Reals have not-a-number, and positive and negative infinity
- Integers single-precision up to ± 2 billion
- Integer division rounds (Stan provides warning)
- Real arithmetic is inexact and reals should not be (usually) compared with `==`

Arrays, vectors, and matrices

- Stan separates arrays, matrices, vectors, row vectors
- Which to use?
- Arrays allow most efficient access (no copying)
- Arrays stored first-index major (i.e., 2D are row major)
- Vectors and matrices required for matrix and linear algebra functions
- Matrices stored column-major (memory locality matters)
- Are not assignable to each other, but there are conversion functions

Logical operators

<i>Op.</i>	<i>Prec.</i>	<i>Assoc.</i>	<i>Placement</i>	<i>Description</i>
	9	left	binary infix	logical or
&&	8	left	binary infix	logical and
==	7	left	binary infix	equality
!=	7	left	binary infix	inequality
<	6	left	binary infix	less than
<=	6	left	binary infix	less than or equal
>	6	left	binary infix	greater than
>=	6	left	binary infix	greater than or equal

Arithmetic and matrix operators

<i>Op.</i>	<i>Prec.</i>	<i>Assoc.</i>	<i>Placement</i>	<i>Description</i>
+	5	left	binary infix	addition
-	5	left	binary infix	subtraction
*	4	left	binary infix	multiplication
/	4	left	binary infix	(right) division
\	3	left	binary infix	left division
.*	2	left	binary infix	elementwise multiplication
./	2	left	binary infix	elementwise division
!	1	n/a	unary prefix	logical negation
-	1	n/a	unary prefix	negation
+	1	n/a	unary prefix	promotion (no-op in Stan)
^	2	right	binary infix	exponentiation
'	0	n/a	unary postfix	transposition
()	0	n/a	prefix, wrap	function application
[]	0	left	prefix, wrap	array, matrix indexing

Assignment operators

<i>Op.</i>	<i>Description</i>
=	assignment
+=	compound add and assign
-=	compound subtract and assign
*=	compound multiply and assign
/=	compound divide and assign
.*=	compound elementwise multiply and assign
./=	compound elementwise divide and assign

- these work with all relevant matrix types
 - e.g., `matrix *= matrix;`

Built-in functions

- All built-in **C++ functions and operators**

C math, TR1, C++11, including all trig, pow, and special log1m, erf, erfc, fma, atan2, etc.

- Extensive library of **statistical functions**

e.g., softmax, log gamma and digamma functions, beta functions, Bessel functions of first and second kind, etc.

- Efficient, arithmetically stable **compound functions**

e.g., multiply log, log sum of exponentials, log inverse logit

Built-in matrix functions

- **Basic arithmetic:** all arithmetic operators
- **Elementwise arithmetic:** vectorized operations
- **Solvers:** matrix division, (log) determinant, inverse
- **Decompositions:** QR, Eigenvalues and Eigenvectors, Cholesky factorization, singular value decomposition
- **Compound Operations:** quadratic forms, variance scaling, etc.
- **Ordering, Slicing, Broadcasting:** sort, rank, block, rep
- **Reductions:** sum, product, norms
- **Specializations:** triangular, positive-definite,

Atomic statements

- **Distribution:** `y ~ normal(mu,sigma)` (increments log probability)
- **Increment log density:** `target += lp;`
- **Assignment:** `y_hat = x * beta;`

Block statements

- **Block:** { ...; ...; ...; }

(allows initial local variables)

Control statements

- **For loop:** `for (n in 1:N) ...`
- **While loop:** `while (cond) ...`
- **Conditional:** `if (cond) ...; else if (cond) ...; else ...;`
- **Break:** `break`
- **Continue:** `continue`

Statements with side effects

- **Print:** `print("theta=", theta);`
- **Reject:** `reject("arg to foo must be positive, found y=", y);`

Distribution statement increments target

- A Stan program defines a log posterior
 - typically through log joint and Bayes's rule
- Sampling statements are just “syntactic sugar”
- A shorthand for incrementing the log posterior
- The following define the same* posterior
 - `y ~ poisson(lambda);`
 - `target += poisson_lupdf(y | lambda);`
- * up to a constant
- Sampling statement drops constant terms

Local scope blocks

- `y ~ bernoulli(theta);`

is more efficient with sufficient statistics

```
{  
  real sum_y; // local variable  
  sum_y = 0;  
  for (n in 1:N)  
    sum_y = sum_y + y[n]; // reassignment  
  sum_y ~ binomial(N, theta);  
}
```

- Simpler, but roughly same efficiency:

```
sum(y) ~ binomial(N, theta);
```


User-defined functions

- **functions** (compiled with model)
 - *content*: declare and define general (recursive) functions (use them elsewhere in program)
 - *execute*: compile with model
- Example

```
functions {  
  
    real relative_difference(real u, real v) {  
        return 2 * fabs(u - v) / (fabs(u) + fabs(v));  
    }  
  
}
```

Special user-defined functions

- When declared with appropriate naming, user-defined functions may
 - be used in sampling statements: `real` return and suffix `_lpdf` or `_lpmf`
 - use RNGs: suffix `_rng`
 - use target accumulator: suffix `_lp`

User-defed pdfs and cdfs

- May be used with sampling and PDF/PMF notation
 - **density**: suffix `_lpdf` if variate (first arg) is real
 - **mass**: Suffix `_lpmf` if variate (first arg) is integer

```
functions {  
  real centered_normal_lpdf(real y, real sigma) {  
    return -0.5 * (y / sigma)^2;  
  }  
  
model {  
  y ~ centered_normal(2.5);           // sampling  
  
  target += centered_normal_lpdf(y | 2.5); // pdf notation
```

Target increments

- May access target or use sampling statements
- Only usable in model block
 - must end in `_lp`

```
functions {  
  vector non_center_lp(vector beta_std, real mu, real sigma) {  
    beta_std ~ normal(0, 1);  
    return mu + sigma * beta_raw;  
  }  
parameters {  
  vector[K] beta_std;  
model {  
  vector[K] beta = non_center_lp(beta_std);  
  y ~ normal(x * beta, sigma);  
}
```

RNG functions

- Only usable in generated quantities block
 - must end in `_rng`

```
functions {  
  real centered_normal_rng(real sigma) {  
    return normal_rng(0, sigma);  
  }  
}
```

```
generated quantities {  
  real alpha = centered_normal_rng(2.7);  
}
```

Under Stan's Hood

Euclidean Hamiltonian Monte Carlo

- **Phase space:** q position (parameters); p momentum
- **Posterior density:** $\pi(q)$
- **Mass matrix:** M
- **Potential energy:** $V(q) = -\log \pi(q)$
- **Kinetic energy:** $T(p) = \frac{1}{2} p^\top M^{-1} p$
- **Hamiltonian:** $H(p, q) = V(q) + T(p)$
- **Diff eqs:**

$$\frac{dq}{dt} = + \frac{\partial H}{\partial p}$$

$$\frac{dp}{dt} = - \frac{\partial H}{\partial q}$$

Leapfrog integrator

- Solves Hamilton's equations by **simulating dynamics**
(symplectic [volume preserving]; ϵ^3 error per step, ϵ^2 total error)
- Given: **step size** ϵ , **mass matrix** M , **parameters** q
- **Initialize kinetic** energy, $p \sim \text{Normal}(0, \mathbf{I})$

- **Repeat** for L leapfrog steps:

$$p \leftarrow p - \frac{\epsilon}{2} \frac{\partial V(q)}{\partial q} \quad [\text{half step in momentum}]$$

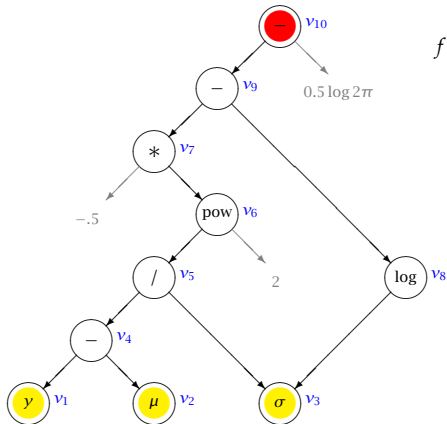
$$q \leftarrow q + \epsilon M^{-1} p \quad [\text{full step in position}]$$

$$p \leftarrow p - \frac{\epsilon}{2} \frac{\partial V(q)}{\partial q} \quad [\text{half step in momentum}]$$

Reverse-mode automatic differentiation

- Eval gradient in (usually small) multiple of function eval time
 - independent of dimensionality
 - time proportional to number of expressions evaluated
- Result accurate to machine precision (cf. finite diffs)
- Function evaluation builds up **expression tree**
- Dynamic program propagates **chain rule** in reverse pass
- Reverse mode computes ∇g in one pass for a function $f : \mathbb{R}^N \rightarrow \mathbb{R}$

Autodiff Expression Graph



$$f(y, \mu, \sigma)$$

$$= \log(\text{Normal}(y|\mu, \sigma))$$

$$= -\frac{1}{2} \left(\frac{y-\mu}{\sigma} \right)^2 - \log \sigma - \frac{1}{2} \log(2\pi)$$

$$\frac{\partial}{\partial y} f(y, \mu, \sigma)$$

$$= -(y - \mu) \sigma^{-2}$$

$$\frac{\partial}{\partial \mu} f(y, \mu, \sigma)$$

$$= (y - \mu) \sigma^{-2}$$

$$\frac{\partial}{\partial \sigma} f(y, \mu, \sigma)$$

$$= (y - \mu)^2 \sigma^{-3} - \sigma^{-1}$$

Autodiff Partials

<i>var</i>	<i>value</i>	<i>partials</i>
v_1	y	
v_2	μ	
v_3	σ	
v_4	$v_1 - v_2$	$\partial v_4 / \partial v_1 = 1$ $\partial v_4 / \partial v_2 = -1$
v_5	v_4 / v_3	$\partial v_5 / \partial v_4 = 1 / v_3$ $\partial v_5 / \partial v_3 = -v_4 v_3^{-2}$
v_6	$(v_5)^2$	$\partial v_6 / \partial v_5 = 2v_5$
v_7	$(-0.5)v_6$	$\partial v_7 / \partial v_6 = -0.5$
v_8	$\log v_3$	$\partial v_8 / \partial v_3 = 1 / v_3$
v_9	$v_7 - v_8$	$\partial v_9 / \partial v_7 = 1$ $\partial v_9 / \partial v_8 = -1$
v_{10}	$v_9 - (0.5 \log 2\pi)$	$\partial v_{10} / \partial v_9 = 1$

Autodiff: Reverse Pass

<i>var</i>	<i>operation</i>	<i>adjoint</i>	<i>result</i>
$a_{1:9}$	$=$	0	$a_{1:9} = 0$
a_{10}	$=$	1	$a_{10} = 1$
a_9	$+=$	$a_{10} \times (1)$	$a_9 = 1$
a_7	$+=$	$a_9 \times (1)$	$a_7 = 1$
a_8	$+=$	$a_9 \times (-1)$	$a_8 = -1$
a_3	$+=$	$a_8 \times (1/v_3)$	$a_3 = -1/v_3$
a_6	$+=$	$a_7 \times (-0.5)$	$a_6 = -0.5$
a_5	$+=$	$a_6 \times (2v_5)$	$a_5 = -v_5$
a_4	$+=$	$a_5 \times (1/v_3)$	$a_4 = -v_5/v_3$
a_3	$+=$	$a_5 \times (-v_4v_3^{-2})$	$a_3 = -1/v_3 + v_5v_4v_3^{-2}$
a_1	$+=$	$a_4 \times (1)$	$a_1 = -v_5/v_3$
a_2	$+=$	$a_4 \times (-1)$	$a_2 = v_5/v_3$

Stan's reverse-mode autodiff

- Easily extensible **object-oriented** design
- **Code nodes** in expression graph for primitive functions
 - requires **partial derivatives**
 - built-in flexible abstract base classes
 - **lazy evaluation** of chain rule saves memory
- Autodiff through templated C++ functions
 - templating on each argument avoids excess promotion

Stan's reverse mode (cont.)

- Arena-based **memory management**
 - specialized C++ operator `new` for reverse-mode variables
 - custom functions inherit memory management through base
- Nested application to support ODE solver

Forward-mode automatic differentiation

- Evaluates expression graph forward from one independent variable to any number of dependent variables
- Function evaluation propagates **chain rule** forward
- In one pass, computes $\frac{\partial}{\partial x} f(x)$ for a function $f : \mathbb{R} \rightarrow \mathbb{R}^N$
 - derivative of N outputs with respect to a single input

Stan's forward mode

- Templated scalar type for value and tangent
 - allows higher-order derivatives
- Primitive functions propagate derivatives
- No need to build expression graph in memory
 - much less memory intensive than reverse mode
- Autodiff through templated functions (as reverse mode)

Second-order derivatives

- Compute Hessian (matrix of second-order partials)

$$H_{i,j} = \frac{\partial^2}{\partial x_i \partial x_j} f(x)$$

- Required for Laplace covariance approximation (MLE)
- Required for curvature (Riemannian HMC)
- Nest reverse-mode in forward for **second order**
- N forward passes: takes gradient of derivative
- All higher-order derivative patterns available through **Stan Playground**.

Third-order derivatives

- Required for Riemannian HMC
- Gradients of Hessians (tensor of third-order partials)

$$\frac{\partial^3}{\partial x_i \partial x_j \partial x_k} f(x)$$

- N^2 forward passes: gradient of derivative of derivative

Third-order derivatives (even more esoteric)

- Gradient of trace of Hessian times matrix
 - $\nabla \text{tr}(H M)$, or
 - needed for Riemannian Hamiltonian Monte Carlo
 - computable in quadratic time for fixed M

Jacobians

- Assume function $f : \mathbb{R}^N \rightarrow \mathbb{R}^M$
- Partial derivatives for multivariate function (matrix of first-order partials)

$$J_{i,j} = \frac{\partial}{\partial x_i} f_j(x)$$

- Required for stiff ordinary differential equations
 - differentiate is coupled sensitivity autodiff for ODE system
- Two execution strategies
 1. Multiple reverse passes for rows
 2. Forward pass per column (required for stiff ODE)

Autodiff functionals

- Functionals map templated functors to derivatives
 - fully encapsulates and hides all autodiff types
- Autodiff functionals supported
 - gradients: $\mathcal{O}(1)$
 - Jacobians: $\mathcal{O}(N)$
 - gradient-vector product (i.e., directional derivative): $\mathcal{O}(1)$
 - Hessian-vector product: $\mathcal{O}(N)$
 - Hessian: $\mathcal{O}(N)$
 - gradient of trace of matrix-Hessian product: $\mathcal{O}(N^2)$
(for SoftAbs RHMC)

Parsing and compilation

- Stan code **parsed** to abstract syntax tree (AST)
(Boost Spirit Qi, recursive descent, lazy semantic actions)
- C++ model class **code generation** from AST
(Boost Variant)
- C++ code **compilation**
- **Dynamic linking** for RStan, PyStan

Coding probability functions

- **Vectorized** to allow scalar or container arguments
(containers all same shape; scalars broadcast as necessary)
- Avoid **repeated computations**, e.g. $\log \sigma$ in

$$\begin{aligned}\log \text{Normal}(y|\mu, \sigma) &= \sum_{n=1}^N \log \text{Normal}(y_n|\mu, \sigma) \\ &= \sum_{n=1}^N -\log \sqrt{2\pi} - \log \sigma - \frac{y_n - \mu}{2\sigma^2}\end{aligned}$$

- recursive **expression templates** to broadcast and cache scalars, generalize containers (arrays, matrices, vectors)
- **traits** metaprogram to **drop constants** (e.g., $-\log \sqrt{2\pi}$ or $\log \sigma$ if constant) and calculate intermediate and return types